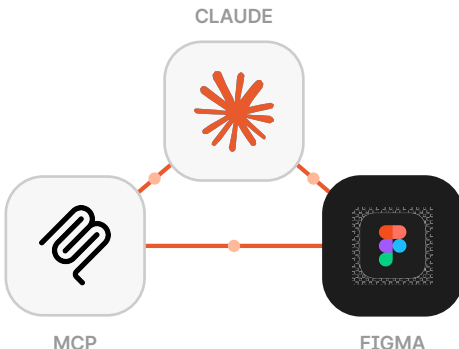


v0.5.0 EDITION

DESIGN SYSTEM AUTOMATION · COMPLETE GUIDE

Harness Core

사용 매뉴얼



Figma MCP × shadcn/ui 기반 디자인 시스템 자동화 하네스

Token

Component

Wireframe

QA

Eval

2026년 6월 발행
Harness Core Project

TABLE OF CONTENTS

목차

PART	장	제목	쪽
01 시작하기	01	무엇인가, 무엇이 아닌가	2
	02	5분 안에 시작하기	5
02 따라하기	03	실전 따라하기 — /init부터 첫 컴포넌트까지	7
03 결과물	04	실제 프로젝트 결과물	15
04 일상 사용법	05	컴포넌트·와이어·QA·Eval·Figma 트랙	17
05 막힐 때	06	자주 막히는 지점	25
	07	구조 더 깊이 보기	27
06 한계	08	한계와 미해결	31
부록 설치·연결	A1	Claude Code 설치	33
	A2	Figma MCP 연결	35

01

INTRODUCTION

무엇인가, 무엇이 아닌가

배경

Harness Core는 새 프로젝트를 시작할 때마다 반복하던 디자인 시스템 셋업을 한 번에 끝낼 수 있게 만든 메타 템플릿이다. 디자인 토큰 정의, 컴포넌트 래핑, 접근성 검증, 와이어프레임 생성까지 — 각 작업이 별도 에이전트, 스킬, 훅으로 분리되어 Claude Code 안에서 자동으로 돌아간다.

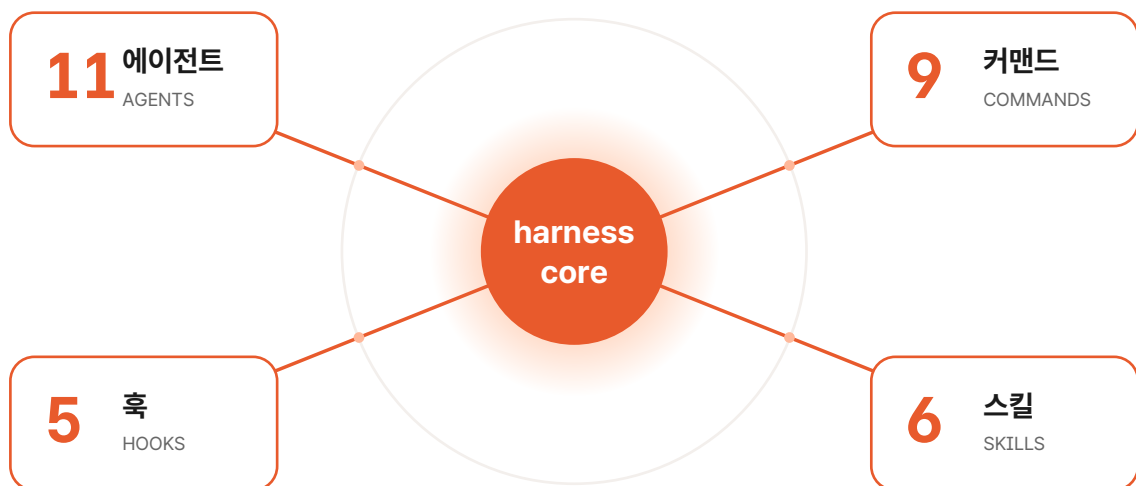


FIG 0 · WHAT'S INSIDE

harness-core는 11개 에이전트, 9개 커맨드, 5개 훅, 6개 스킬로 구성된다 (v0.5.0 기준)

v0.3.3 → v0.5.0, 무엇이 바뀌었나

버전	핵심	한 줄 요약
v0.3.3	가드 정합성 수정	혹이 Edit에서 안 돌던 배선 누락 수정 + 회귀 테스트 14→26
v0.4.0	write 가능 Figma 빌더	Figma에 컴포넌트·토큰을 실제로 생성 (MCP write)
v0.5.0	Figma QA 커버리지	Figma 화면 리뷰 + Figma↔코드 스크린샷 비교

KEY INSIGHT

v0.3.3까지 core의 전제는 "Figma MCP는 read-only"였다. v0.4부터 이 전제가 조건부로 바뀐다 — write 가능한 Figma 플러그인 MCP(use_figma)가 붙어 있으면 Figma에 직접 생성하고, 없으면 기존 read-only 동작(스펙시트·비교)으로 폴백한다. 같은 하네스가 두 환경을 모두 커버한다.

KEY INSIGHT

디자인 시스템 자체가 아니다. 디자인 시스템을 만들고 유지하는 도구다. 프로젝트마다 PROJECT.md 한 장을 채우면 그 프로젝트만의 컬러 스케일, 타이포, 컴포넌트가 자동 생성된다.

누가 쓰면 좋은가

디자이너 없이 혼자 또는 소수가 프로젝트를 만드는 개발자에게 가장 맞다. 이미 디자인팀이 정립된 디자인 시스템을 운영 중이라면, 이 도구의 자동화는 오히려 거추장스럽다.

Figma 시안 없이도 작동한다. 단, Figma 파일 키가 있으면 토큰을 양방향 동기화할 수 있다.

필요한 환경

OS	macOS 또는 Linux (Windows는 WSL 권장)
Node.js	20 이상
패키지 매니저	pnpm — INSTALL.sh가 자동 설치
에디터	VS Code 또는 Cursor + Claude Code 확장
선택	Figma 유료 계정 + MCP 인증

무엇이 아닌가

오해를 줄이기 위해 명시한다.

디자인 시스템 자체가 아니다

토큰과 컴포넌트는 PROJECT.md를 기반으로 매번 새로 생성된다. 본 패키지에는 미리 만들어진 시스템이 없다.

Figma 플러그인이 아니다

Claude Code 안에서 작동한다. Figma MCP는 옵션이며, 없어도 모든 기능이 동작한다.

디자이너 대체가 아니다

출발점을 단단하게 만들 뿐 디자인 결정 자체를 하지는 않는다. 톤과 캐릭터는 사람이 정한다.

범용 코드 생성기가 아니다

디자인 시스템과 그 주변(토큰, 컴포넌트, QA)에 특화되어 있다. 일반 앱 로직은 직접 작성한다.

KEY INSIGHT

톤과 금지 규칙은 선택이지만, 채워둘수록 결과의 "AI가 만든 듯한" 느낌이 줄어든다. 특히 금지 규칙("그림자 사용 최소화", "라디우스 12px 초과 금지")은 디자인의 캐릭터를 만드는 데 가장 영향이 크다.

3. Claude Code에서 첫 명령

VS Code 또는 Cursor에서 폴더를 열고, Cmd+Shift+P → "Claude Code: Open in Terminal"을 실행한다. 대화창에 다음을 입력한다.

```
/init
```

Claude는 PROJECT.md를 읽고, 브랜드 컬러를 OKLCH 11단계로 자동 변환하고, 토큰 파일 7개를 생성한 뒤 DESIGN.md 카탈로그를 작성한다. 전 과정이 한국어로 진행되며 30초 안에 끝난다.

4. 결과 확인

```
pnpm dev
# → http://localhost:5173
pnpm storybook
# → http://localhost:6006
```

Figma 연동 (선택)

Figma 시안과 토큰을 양방향 동기화하려면 MCP 인증이 필요하다. Claude Code 대화창에 /mcp를 입력하고 안내를 따른다. 인증 후 PROJECT.md에 파일 키와 페이지 ID를 적어두면 /ds-token 명령이 자동으로 양쪽을 비교한다. Figma 없이도 모든 기능이 작동한다. 다만 시안 → 코드 변환이 수동이 된다.

자주 묻는 — 시작 단계

Q. INSTALL.sh가 "node not found"라고 한다

Node 20 이상이 필요하다. nodejs.org에서 LTS 버전을 받아 설치한 뒤 다시 시도한다. 이미 설치되어 있다면 node -v로 버전을 확인한다.

Q. Claude Code 확장이 안 보인다

VS Code Marketplace에서 "Claude Code"를 검색해 설치한다. Cursor 사용자도 동일하다.

Q. /init이 PROJECT.md placeholder가 비어있다고 거부한다

{...} 형태로 남은 자리를 모두 채워야 한다. 최소 이름, 브랜드 컬러, 폰트 세 개는 필수다. 톤이나 금지 규칙처럼 선택 항목은 비워도 되지만 중괄호는 지운다.

03

WALKTHROUGH

실전 따라하기 — /init부터 첫 컴포넌트까지

이 장은 처음 쓰는 사람이 이 글만 보고 따라올 수 있게 만든 실전 기록이다. 각 단계를 ① 무엇을 한다 → ② 안에서 어떻게 도는가 → ③ 실제로 뭐가 나오는가 → ④ 왜 하는가 순으로 설명한다. 숫자·출력은 실제 프로젝트(브랜드색 초록 #22c55e)에서 그대로 가져왔다.

| 시작 전에 — 머릿속에 3개만

① 누가 누구를 부르나



FIG 3-1 · CALL CHAIN

옆에서 훑어 감시 — 잘못된 작업이면 차단. 단계는 한 번에 하나, 당신이 다음 신호를 준다.

커맨드(/init 등)는 입구다. 치면 에이전트(실제 일꾼)가 깨어나 일한다. 에이전트는 끝나면 "다음은 이걸 하세요"라고 추천만 한다 — 단계는 한 번에 하나, 당신이 다음 신호를 준다. 자동으로 쪽 안 굴러가는 게 중요한 이유는 중간에 검토·결정할 수 있어야 하기 때문이다.

② "토큰"이 뭐고 왜 중요한가

토큰 = 디자인 값에 붙인 이름. #22c55e 같은 raw 값 대신 --color-primary라는 이름을 쓰는 것이다. 색을 바꿔야 하면 토큰 한 줄만 고치면 그 토큰 쓰는 모든 컴포넌트가 자동으로 바뀐다. 코드·Figma가 같은 이름을 쓰면 디자이너와 개발자가 같은 언어로 말하게 된다. 이름에 접근성·다크모드 규칙을 박아두면, 매번 신경 안 써도 자동으로 지켜진다.

핵심

하드코딩(#22c55e)을 흘뿌리지 않고, 이름(토큰) 하나로 통제하는 것. 그래서 토큰을 먼저 만든다.

③ 두 갈래 트랙

코드 트랙(/ds-component 등)은 코드(src/)를 만든다. Figma 트랙(/build-component-/figma-tokens)은 Figma에 실제로 만든다(write 가능한 Figma MCP 있을 때). 둘은 /ds-token에서 만나 서로 맞춘다. 자세한 다이어그램은 05장.

전체 STEP 한눈에

- STEP 0 · PROJECT.md**
씨앗 심기 — 브랜드색·폰트·톤 (1회)
- STEP 1 · /init**
코드 토큰 생성 + DESIGN.md
- STEP 2 · /figma-tokens**
Figma Variables 실제 생성
- STEP 3 · /ds-token**
코드 ↔ Figma 정합 맞추기
- STEP 4 · /build-component + /ds-component**
Figma 본 + 코드 래퍼
- STEP 5 · /qa-a11y**
접근성 자동 검증

FIG 3-2 · WALKTHROUGH STEPS

왜 이 순서? 컴포넌트는 토큰을 재료로 쓴다. 재료가 없으면 컴포넌트를 못 만든다 — 그래서 토큰이 항상 먼저.

STEP 0 — PROJECT.md (씨앗 심기)

프로젝트당 딱 한 번 채우는 파일. 브랜드색 1개, 폰트, (선택) Figma 키/페이지 ID, 톤.

- 브랜드 컬러 (hex 1개): #22c55e
- 폰트: Pretendard
- Figma 파일 키 / 페이지 ID: (Figma 트랙 쓸 때만)
- 톤: "노션처럼 정돈된 정보 밀도" 같은 구체적 명사구 3~5개

왜

이 한 장이 모든 자동 생성의 씨앗이다. 브랜드색 하나만 넣으면 거기서 전체 색 스케일이 나온다. "사람의 결정"은 여기(=무엇을 만들지)서 끝나고, 이후 "어떻게"는 전부 자동이다. 비어 있으면 에이전트가 "일반 기본값"으로 일해 "AI가 만든 듯한" 결과가 나온다. 톤·금지 규칙을 채우는 게 품질을 가장 크게 바꾼다.

STEP 1 — /init (토큰 생성 + 카탈로그)

| 안에서 어떻게 도는가

/init 커맨드는 곧장 ds-token-builder 에이전트에게 위임한다. 에이전트는 다음 순서로 일한다.

- 1 PROJECT.md 파싱**
브랜드색·폰트·다크모드 여부
- 2 color-scale-builder 스킴**
브랜드색 → OKLCH 50~900 스케일
- 3 neutral + 시맨틱색 생성**
success/warning/danger/info
- 4 src/tokens/*.css 7개 작성**
import 주석 해제
- 5 WCAG 대비 자동 검증**
미달이면 주석/조정
- 6 DESIGN.md 카탈로그 작성**
에이전트 자동 갱신용
- 7 sentinel 정리 + 보고**
node로 정리 (v0.5.1: rm 안 씀)

FIG 3-3 · /INIT INTERNALS

/init이 안에서 거치는 7단계 — 한 번 돌면 코드 토큰 + 카탈로그가 완성된다

| 실제로 나온 것 (브랜드색 초록 #22c55e)

/init 완료. 토큰 7개 파일 생성, WCAG AA 통과, 타입체크 클린.
- brand: #22c55e → brand-600 베이스 (초록 149.6°, OKLCH 균일)
- success-600 / warning-400(검정 텍스트로 AA 확보) / danger-600 / info-600
- light/dark 양쪽 시맨틱 alias 정의

파일	답는 것
colors.css	brand 50~900, neutral 0~900, semantic alias(--color-primary 등), light/dark
typography.css	--font-sans, --font-size-*
spacing.css	--space-* (4px 그리드)
radius.css	--radius-*

motion.css	duration-easing
semantic.css	focus-ring-shadow-z-index
index.css	위 전부 import

왜 이렇게 (핵심)

brand 1개 → 전체 스케일: 50~900을 손으로 안 고른다. OKLCH로 밝기 단계가 균일하게 계산돼, 어디에 써도 톤이 고르다.

raw vs semantic alias 2층 구조: raw는 --color-brand-500(실제 색), alias는 --color-primary → brand-600을 가리킨다. 컴포넌트는 alias만 쓴다. 그래서 "primary를 한 단계 어둡게" 같은 변경이 alias 한 줄로 끝난다.

WCAG 자동 검증: 초록 brand 위 흰 텍스트가 대비 부족하면 primary를 brand-600(한 단계 어둡게)으로 자동 선택했다. warning(노랑)은 검정 텍스트라야 대비가 나오니 그렇게 잡았다 — 접근성을 사람이 매번 계산 안 해도 된다.

DESIGN.MD는 건드리지 마라

DESIGN.md는 에이전트가 자동 갱신하는 카탈로그다. 직접 편집하면 다음 실행 때 덮어써진다. 정의를 바꾸려면 PROJECT.md를 고친다. 확인 팁: pnpm eval을 돌리면 토큰 대비가 CRITICAL 0건인지 점수로 나온다.

STEP 2 — /figma-tokens (Figma에 토큰 실제 생성)

방금 만든 코드 토큰을 Figma Variables로 실제로 만든다. write Figma MCP 필요. 담당은 ds-figma-token-builder.

왜

디자이너는 Figma에서 작업한다. 코드의 토큰과 이름이 1:1로 같은 변수를 Figma에도 두면(--color-brand-500 ↔ brand/500), 둘이 같은 언어가 된다. 이후 컴포넌트를 Figma에서 그릴 때 raw 색을 찍지 않고 이 변수를 바인딩한다 → 코드와 안 어긋난다.

실제로 나온 것

컬렉션 4개 / 신규 변수 83개

- Colors / Typography / Spacing / Radius
- swatch 시각 검증 PASS (Light 356:2 / Dark 360:2)

주목: Figma에 기존 brand가 주황(#f97316)으로 잘못 들어가 있던 걸 PROJECT.md 기준 초록(#22c55e)으로 재기록.

이 "주황 → 초록" 보정이 바로 다음 단계가 필요한 이유다 — 코드와 Figma가 어긋나 있었다(드리프트).

STEP 3 — /ds-token (코드 ↔ Figma 정합 맞추기)

코드 토큰과 Figma 변수를 양방향 비교해서 어긋난 곳을 분류하고, 사람이 결정해 맞춘다.

분류	뜻	조치
일치	양쪽 같음	무시
Figma에만	코드에 없음	추가할지 묻는다
코드에만	Figma에 없음	추가할지 / 폐기인지
값 충돌	양쪽 있는데 값 다름	사람이 결정

실제로 나온 것

처음엔 충돌 33건(신호색 success/warning/danger + radius). 결정 기준은 두 가지였다 — 색·대비 토큰은 검증된 코드(OKLCH) 채택(코드는 /init에서 WCAG 통과한 값), radius 같은 미적 토큰은 의도된/풍부한 쪽 채택(xs=2px 추가, 2xl 20→24px). 재검증 결과 충돌 0건 — 색 70/70 일치, radius 8/8, font-size 6/6, 코드전용 12개는 의도된 것.

왜 충돌이 "정상"인가

/ds-token의 존재 이유가 "어긋난 곳을 찾는 것"이다. 충돌이 났다 = 드리프트를 제대로 감지했다는 뜻. 위험한 건 충돌이 안 뜨고 조용히 어긋난 채 쓰는 것이다. 처음엔 코드와 Figma가 따로 만들어져서 많이 갈리지만, 한 번 맞추고 /figma-tokens 재실행하면 이후엔 대부분 "일치"로 조용해진다. = 최초 1회 정합 작업.

STEP 4 — 컴포넌트 (/build-component + /ds-component)

/build-component Button은 Figma에 Component Set + 문서를, /ds-component Button은 코드에 cva 래퍼 + 스토리를 만든다.

실제로 나온 것 (Button)

Figma: Button Component Set (intent 4 × size 3 × state 3 = 36 variants)
+ Dark Preview, Light/Dark 시각 검증 PASS
모든 컬러·라디우스·타이포·스페이싱이 Figma Variables에 바인딩
코드 : src/components/Button/ (cva 래퍼 + 스토리), DESIGN.md 갱신
(참고: Pretendard가 Figma에 없어 Inter로 풀백 — 폰트 업로드 시 교체)

왜 둘 다 만드나

Figma 본은 디자이너가 보는 진실, 코드 래퍼는 개발이 쓰는 진실. 둘의 variant 이름을 일치시켜 어긋남을 막는다. 보통 /build-component(디자인) → 확인 → /ds-component(코드) 순이다. 코드 먼저 만들면 그 cva 구조를 Figma가 미러링하게 할 수도 있다. 둘 다 STEP 1~3에서 만든 토큰을 alias로 쓴다 — 그래서 토큰을 먼저 만든 것이다.

STEP 5 — /qa-a11y (접근성 검증)

컴포넌트의 모든 색 조합 + 다크모드 동등성 + ARIA/키보드를 WCAG 2.2 AA로 검사한다.

A11y 32/32 PASS — CRITICAL 0, 다크 동등성 위반 0
primary 5.32 / 6.49 (L/D), danger 6.66 / 5.55, secondary text 18.1 / 17.34 ...

왜

브랜드색이 초록으로 바뀌면 대비가 달라진다. 눈으로 멀쩡해 보여도 대비가 미달일 수 있다(토스 블루 #3182f6 위 흰 텍스트가 3.71:1로 AA 미달인 것처럼). QA가 이런 함정을 전부 잡고, 미달이면 OKLCH 조정안까지 제시한다.

전체 플로우 한눈에 — 왜 이 순서



FIG 3-4 · END-TO-END

씨앗(PROJECT.md) → 코드 토큰 → Figma 토큰 → 정합 → 컴포넌트(Figma+코드) → 접근성 검증

왜 토큰 → 컴포넌트 → 화면 순서인가? 컴포넌트는 토큰(색·간격·라디우스)을 재료로 쓴다. 재료가 없으면 컴포넌트를 못 만든다. 그래서 토큰이 항상 먼저다.

자주 나오는 "왜?" 모음

Q. 왜 #22c55e를 직접 안 쓰고 토큰을 쓰나?

색 한 번 바꾸려면 코드 전체를 뒤져야 하기 때문. 토큰이면 한 줄로 끝나고, 혹이 하드코딩을 아예 차단한다.

Q. 왜 raw 말고 alias(--color-primary)를 컴포넌트에서 쓰나?

"primary를 어둡게" 같은 의미 단위 변경을 alias 한 줄로 하기 위해. raw를 직접 쓰면 그 색을 쓰는 곳을 다 고쳐야 한다.

Q. 왜 충돌이 그렇게 많이 났나? 잘못된 건가?

아니다. 코드와 Figma가 따로 만들어졌다 처음 맞대면해서 그렇다. 도구가 드리프트를 제대로 잡은 것 = 건강한 신호. 최초 1회만 많고 이후엔 조용하다.

Q. 왜 한 번에 다 자동으로 안 돌아가나?

중간에 사람이 결정할 게이트(색 충돌, 브랜드 드리프트, QA CRITICAL)가 있어서. 자동조종이면 그걸 멋대로 잘못 결정한다. 그래서 한 단계씩 + 게이트에서 멈춤이 의도된 설계다.

Q. Figma 없어도 되나?

된다. /init·/ds-component·/qa·/wireframe·pnpm eval은 Figma 없이 전부 작동한다. Figma 트랙(/figma-tokens·/build-component)만 write MCP가 있을 때 돈다.

이 장은 실전 1회차(브랜드색 초록) 기록을 바탕으로 한다. 숫자는 프로젝트마다 다르지만 순서와 이유는 동일하다.

04

REAL OUTPUT

실제 프로젝트 결과물

추상적인 설명만으로는 감이 안 잡힐 수 있다. 한 여행 앱 프로젝트에서 Harness Core로 실제 만들어진 산출물을 흐름대로 보면 감이 잡힌다. PROJECT.md 한 장에서 시작해 토큰, 컴포넌트, 와이어프레임, Figma 통합까지 어떻게 이어지는지 한눈에 보인다.



FIG 00 · PIPELINE

PROJECT.md 한 장에서 토큰 → 컴포넌트 → 와이어프레임 → QA 까지 이어지는 흐름

디자인 토큰 — 한 컬러에서 자동 생성

PROJECT.md에 브랜드 컬러 hex 하나만 적으면 50~950 스케일이 자동 생성된다. Typography와 시맨틱 컬러(success/warning/danger)도 함께. Storybook에서 즉시 확인 가능하다.

Figma 통합 — 코드와 1:1 매칭

ds-figma-component-builder가 생성한 컴포넌트들이 Figma 에셋 패널에 그대로 나타난다. 디자이너와 개발자가 같은 컴포넌트를 본다. write 가능한 Figma MCP가 연결돼 있으면 /build-component로 직접 생성되고, read 전용 환경이면 스펙시트를 받아 수동으로 만든다(8장 참조).

Storybook 카탈로그 — 한 화면에서

/ds-component 한 번이면 Storybook 스토리도 함께 만들어진다. 한 컴포넌트당 5개 안팎(Default·Variants·Sizes·States·Matrix).

와이어프레임 → 디자인 진화

Welcome 화면은 와이어프레임 단계에서 시작하고, Login 화면은 같은 와이어프레임 구조 위에 디자인 토큰을 입혀 완성된다. 회색에서 컬러로 가는 단계가 자연스럽다.

이 사례에서 배울 점

이 모든 산출물의 공통점은 PROJECT.md 한 장에서 시작했다는 것이다. 브랜드 컬러 한 개, 폰트 한 개, 톤 세 개. 그 외 모든 토큰·컴포넌트·스토리·Figma 통합은 자동이다. 사람의 결정이 필요한 곳은 "무엇을 만들지"이지, "어떻게 만들지"가 아니다.

05

DAILY WORKFLOW

일상 사용법

한 번 셋업이 끝나면 일상은 단순하다. 컴포넌트 추가, 와이어프레임 그리기, QA 돌리기. 거의 모든 작업은 슬래시 명령 한 줄이다.

v0.4부터 작업은 **코드 트랙**과 **Figma 트랙** 두 갈래로 나뉜다. 코드에서 시작하든(/ds-component) Figma에서 시작하든(/build-component) 결국 /ds-token이 둘을 맞춘다. 어느 쪽이 "진실의 원천"인지는 프로젝트가 정한다. Figma 트랙은 write 가능한 Figma MCP가 있을 때만 동작하고, 없으면 폴백한다.

명령어 한눈에

명령	트랙	역할	전제
/init	코드	프로젝트 부트스트랩 — 토큰 생성 (1회)	—
/ds-component <이름>	코드	shadcn 받아서 디자인시스템 래퍼(cva) 생성	—
/wireframe <이름>	코드	회색 5단계 와이어프레임 화면 생성	—
/build-component <이름> 	Figma	Figma에 Component Set + Property Table 생성	write MCP
/figma-tokens [종류] 	Figma	Figma에 Variables(멀티모드) 생성	write MCP
/ds-token	동기화	Figma Variables ↔ 코드 토큰 양방향 비교	read MCP
/qa	검증	전체 QA (디자인시스템 + a11y + 코드 와이어)	—
/qa-a11y	검증	접근성만 단독 — 더 빠름	—
/qa-figma-wireframe <화면> 	검증	Figma 화면 frame 검증	read MCP
pnpm eval	회귀	하네스 회귀 추적 — 기준선 대비 점수	—
pnpm test:hooks	회귀	훅 가드 회귀 — 26 케이스 (v0.5)	—

컴포넌트 추가

shadcn/ui 컴포넌트 하나를 받아 디자인시스템에 통합한다.

```
/ds-component Button
```

- 1 **/ds-component Button**
사용자가 커맨드 입력
- 2 **shadcn 원본 설치**
src/components/ui/button.tsx — 손대지 않음 (Bash로 설치)
- 3 **cva 래퍼 생성**
src/components/Button/ — 토큰을 alias로 매핑
- 4 **Storybook 스토리 생성**
Default · Variants · Sizes · States · Matrix
- 5 **WCAG + Eval 자동 검증**
대비 위반 시 OKLCH 조정안 보고, pnpm eval로 회귀 확인

FIG 4-1 · COMPONENT FLOW

/ds-component 한 줄이 거치는 단계 — 원본 설치부터 접근성·회귀 검증까지

Figma 시안 URL을 함께 주면 더 정확하다.

```
/ds-component Card https://figma.com/file/.../node-id=123:456
```

와이어프레임 만들기

디자인보다 정보 구조를 먼저 잡고 싶을 때 쓴다. 회색 5단계만 허용되며, 강조는 typography weight와 size로만 한다. 시맨틱 컬러 사용은 흑이 자동 차단한다 — Write든 Edit든 동일하게(v0.3.3에서 Edit 검사 누락이 수정됨).

```
/wireframe Login
```

명세가 모호하면 Claude가 한 번 질문한다 — "이메일/비밀번호만? 소셜 로그인도?" 같은 식이다. 답을 주면 src/wireframes/Login/Login.tsx에 화면이 생성되고 Storybook에서 mobile/desktop 두 viewport로 확인할 수 있다.

WIREFRAME 원칙

와이어프레임에 색을 "잠깐만" 쓰고 싶은 유혹이 든다면, 그건 와이어프레임이 끝나고 디자인 단계로 넘어갈 신호다. 회색만으로 정보 위계가 잡히지 않으면 디자인을 입혀도 잡히지 않는다.

QA 돌리기

토큰 정합성, 접근성, 와이어 UX를 한 번에 검사한다.

```
/qa  
/qa Button # 특정 컴포넌트만
```

결과는 docs/qa-reports/ 폴더에 마크다운 리포트로 저장된다. CRITICAL 이슈가 있으면 첫 줄에 빨간색으로 표시된다.

토스 블루(#3182f6) 위에 흰 텍스트는 시각적으로 멀쩡해 보이지만 실제 대비비는 3.71:1로 AA 기준 미달이다. QA가 이런 함정을 모두 잡아낸다.

— 실제 검증 사례

Eval 돌리기

v0.3에서 추가됐다. /qa가 "이 컴포넌트가 잘 만들어졌나"를 본다면, Eval은 "하네스 자체가 잘 작동하나, 퇴화하지 않았나"를 점수로 본다. 검사 대상이 다르다 — QA는 결과물을, Eval은 그 결과물을 만드는 도구를 본다.

QA		Eval
대상	만들어진 컴포넌트	하네스(도구)의 산출물 정합성
질문	이거 잘 만들어졌나?	도구를 믿을 수 있나?
출력	통과 / 실패	점수 + 기준선 대비 변화
잡는 것	지금 이 결과물의 결함	도구의 실력 변화·퇴화

KEY INSIGHT

식당으로 치면 QA는 접시 하나하나 맛보는 것이고, Eval은 주방 시스템 전체를 점검하는 것이다. 접시를 아무리 검사해도 레시피 자체가 가끔 짜게 나오는 구조라면 못 잡는다. Eval은 그 레시피(=하네스)를 평가한다.

에이전트를 재호출하지 않아 API 비용이 0이고 1초 안에 끝난다. 산출물(토큰·컴포넌트·와이어프레임)을 직접 정적 스캔해 채점하며, 대비 검사는 기존 a11y-contrast-checker 엔진을 그대로 재사용한다 — 색 계산이 둘로 갈라지지 않게.

기준선으로 회귀 잡기

```
pnpm eval:baseline # 지금 상태를 "정상" 기준선으로 (최초 1회)
pnpm eval          # 채점 → 기준선 대비 ▲/▼ 변화 표시
```

점수가 기준선보다 떨어지면 "이번 변경이 하네스를 퇴화시켰을 수 있음"을 경고한다. CRITICAL이 있으면 종료 코드 1을 반환해 CI 빌드를 막는다. high 가중치 검사(대비 등)가 토큰 부재로 실행되지 못하면 만점 대신 "결과 불완전"으로 표시되어, 검사를 못 한 채 안심하는 일이 없다.

v0.2에서 잡은 CRITICAL 5건(danger 버튼 대비 등)이 그대로 Eval 태스크의 출발점이 됐다. 이미 터진 문제에서 시작한다는 EDD(Eval 주도 개발)의 원칙이다.

— 설계 노트

흑 가드 회귀 테스트 (v0.3.3 → v0.5)

흑 자체를 수정했거나 가드 동작이 의심되면 흑 회귀 테스트를 돌린다. 모든 흑을 실제 Claude Code 입력 형태(Write-Edit-MultiEdit)로 먹여, 가드가 정말 차단하는지 26개 케이스로 검증한다.

```
pnpm test:hooks # 26 pass — 흑 동작 + settings.json matcher 배선 검증
```

왜 14에서 26으로 늘었나

v0.3.3에서 흑 코드는 Edit/MultiEdit를 읽도록 고쳤지만, settings.json의 PostToolUse matcher가 "Write"뿐이라 컴포넌트를 Edit으로 수정할 때 a11y 가드가 통째로 우회됐다. 회귀 테스트는 흑을 직접 호출해 matcher 게이트를 건너뛰어 통과해버려 "거짓 안심"을 줬다. 그래서 matcher를 "Write|Edit|MultiEdit"로 명시하고, 테스트가 settings.json 배선까지 검증하도록 확장했다 — "흑 코드는 Edit를 읽는데 matcher는 Write뿐" 같은 불일치를 이제 테스트가 잡는다.

Figma 토큰 생성 — /figma-tokens (v0.4 신규)

코드 토큰을 Figma Variables로 실제로 만든다. 담당 에이전트는 ds-figma-token-builder. core의 토큰 명령 3종은 방향이 다르니 헷갈리지 말 것 — /init은 브랜드색을 코드 토큰으로, /ds-token은 코드↔Figma 양방향 비교(read), /figma-tokens는 Figma에 Variables를 write 생성한다.

```
/figma-tokens 전체
# 1 DESIGN.md/PROJECT.md + (권장) src/tokens/*.css 읽어 명명 일치
# 2 whoami 인증·플랜 확인 (Starter=모드1, Pro+=multi-mode)
# 3 기존 컬렉션·변수 조회 (같은 이름은 수정, 중복 금지)
# 4 use_figma로 변수 생성 (컬렉션 1개=1종류, ~24개씩 분할)
# 5 swatch/preview frame 생성 + 6 get_screenshot 시각 검증
```

Figma Variable 명명은 코드 CSS와 1:1로 맞춘다.

코드 (CSS)	Figma Variable	비고
--color-brand-500	brand/500	스케일 50~900
--color-primary / --color-danger	primary / danger	시맨틱 alias 우선
--space-md	space/md	gap/padding
--radius-md	radius/md	corner

--font-size-base

font-size/base

타이포

컬렉션은 Colors / Typography / Spacing / Radius로 1컬렉션=1종류. 스코프는 명시 필수(ALL_SCOPES 금지).
모드는 Brand는 Light/Dark 동일, Neutral은 반전, Semantic은 한 단계 밝게.

Figma 컴포넌트 생성 — /build-component (v0.4 신규)

Figma에 Component Set과 Property Table 문서까지 실제로 만든다. 담당은 ds-figma-component-builder.
산출물은 항상 두 개가 함께 나온다 — 개발자가 instance로 쓰는 Component Set(variants 격자)과, 축 라벨·점선 격자·셀별 instance·Ready-made examples로 구성된 — Documentation frame.

```
/build-component Badge
# 1 DESIGN.md/PROJECT.md 읽기 (페이지 ID·토큰·컨벤션)
# 2 use_figma 호출 시 skillNames "figma-use,figma-generate-library" 로드
# 3 기존 컴포넌트·변수 조회 (같은 이름 있으면 수정 모드)
# 4 (선택) src/components/{Name}/ cva variants 참조 → 이름 일치
# 5 incremental: default 1개 → clone로 variants → combineAsVariants
# 6 토큰 alias 바인딩 (raw 값 금지) + 7 Property Table 생성
# 8 get_screenshot 시각 검증 (Set + Documentation 둘 다)
```

포팅하면서 보존한 정밀 규칙: 신규 frame은 기존 frame들의 $\max(x+width)+200$ 에 배치(좌표 충돌 방지). Figma number variable은 PIXELS로 해석하므로 line-height는 변수 바인딩 금지하고 {unit:"PERCENT", value:120/150/175}로 직접 입력. Component Set은 Components 페이지, 문서는 Documentation 페이지에 분리. 코드(src/)와 .claude/는 절대 안 건드리고 Figma만 write.

KEY INSIGHT

/build-component는 Figma를 만들고, /ds-component는 코드를 만든다. 보통
/build-component Badge → (디자인 확인) → /ds-component Badge 순으로 이어, Figma
본과 코드 래퍼의 variant 이름을 일치시킨다.

Figma 화면 QA — /qa-figma-wireframe (v0.5 신규)

core는 그동안 코드 와이어(src/wireframes/)만 검증했다. v0.5부터 Figma에 그려진 화면 frame도 검증한다.
담당은 qa-figma-wireframe. 대상이 다르니 구분할 것 — /qa wireframe은 코드 React 화면을,
/qa-figma-wireframe은 Figma frame을 read-only MCP로 본다.

```
/qa-figma-wireframe Login
# frame get_metadata + get_design_context + get_screenshot 수집
# Critical/High/Medium/Low 체크리스트 검증
# docs/qa-reports/QA-Figma-Wireframe-{화면}-{날짜}.md 작성
```

등급

검사 항목

Critical	레이아웃 깨짐 없음 · line-height PERCENT · 모바일 SafeArea · child가 frame 안
High	CTA 1개 명확 · 1화면 1메시지 · 타이포 위계 · 아이콘 버튼 접근 이름
Medium	instance variant 적절 · 간격 토큰(space/*) · 카피 톤 · 비표준 컴포넌트 X
Low	다크모드 · 여백 균형 · placeholder

/qa 보강 — Figma↔코드 스크린샷 비교 (v0.5)

qa-design-system의 Figma 정합성 단계가 노드 이름 매칭에서 실제 렌더 비교로 깊어졌다. PROJECT.md에 Figma 키가 있으면, Figma 원본 get_screenshot과 코드 렌더(Storybook 캡처)를 나란히 비교해 컬러·간격·라운드·그림자·정렬의 실제 차이를 "Figma vs 코드" 표와 스크린샷 2장으로 리포트한다. Figma 키나 MCP가 없으면 이 단계는 건너뛰고 코드 구조 검사만 진행한다.

토큰 동기화

디자이너가 Figma Variables를 수정한 뒤 코드와 일치시키거나, 코드의 새 토큰을 Figma에 반영해야 할 때 쓴다.

/ds-token	
분류	조치
일치	양쪽 같음 — 무시
Figma에만	코드에 추가할지 묻는다
코드에만	Figma에 추가해야 하는지 또는 deprecated인지 확인
값 충돌	양쪽에 있지만 값이 다름 — 사용자가 결정

/ds-token 자체는 read 기반 양방향 비교다. 코드의 새 토큰을 Figma에 실제로 만들려면 write 트랙의 /figma-tokens를 쓴다. write MCP가 없으면 /ds-token은 디자이너에게 전달할 변경 사항을 텍스트로 출력하는 폴백으로 동작한다.

DESIGN.MD는 건드리지 마라
DESIGN.md는 에이전트가 자동으로 갱신하는 카탈로그다. 직접 수정하면 다음 /init이나 /ds-component 실행 시 덮어쓰여진다. 프로젝트 정의를 바꾸고 싶으면 PROJECT.md를 수정하라.

일상의 흐름 예시 (Figma 트랙 포함)

write MCP가 붙어 있을 때, 새 디자인 시스템 요소를 만드는 전형적인 흐름이다.

- 1 **PROJECT.md 작성**
브랜드색·폰트·Figma키·톤 (프로젝트마다 1회)
- 2 **/init**
코드 토큰 + DESIGN.md 생성
- 3 **/figma-tokens 전체**
Figma에 Variables 생성 (write MCP)
- 4 **/build-component Badge**
Figma에 컴포넌트 + 문서 생성 (write MCP)
- 5 **/ds-component Badge**
코드 래퍼(cva) + 스토리
- 6 **/ds-token · /qa · /qa-figma-wireframe**
동기화 + 검증
- 7 **self-check → pnpm eval**
하네스 회귀 점검

06

TROUBLESHOOTING

자주 막히는 지점

여기 모은 항목은 실제로 자주 막히는 곳이다. 처음 사용할 때 한 번씩 훑어보길 권한다.

토큰 파일을 직접 수정하려는데 혹은 차단한다

src/tokens/ 폴더는 보호되어 있다. 정공법은 /ds-token이나 /init을 다시 실행하는 것이다 — 그 명령이 sentinel을 자동으로 만들고(node로 생성) 작업 후 지운다. 수동으로 잠깐 열어야 한다면 sentinel 파일을 직접 만든다.

v0.3.3부터 rm은 전면 차단되므로 sentinel 생성·삭제는 node로 한다.

```
# 수동으로 5분간 열기
node -e "require('fs').writeFileSync('.claude/.ds-token-active', '')"
# 작업 후 닫기
node -e "require('fs').rmSync('.claude/.ds-token-active', {force:true})"
```

shadcn 원본을 수정하고 싶다

src/components/ui/ 폴더의 파일은 손대지 않는 게 원칙이다. shadcn 업데이트 시 덮어써지고, 그동안 한 수정이 사라진다. 대신 같은 이름의 사용자 래퍼(src/components/{이름}/{이름}.tsx)에서 모든 커스터마이징을 한다. (이 폴더는 settings.json의 deny가 직접 보호하며, shadcn은 Bash로 설치되어 혹은 거치지 않는다.)

/qa에서 CRITICAL이 나왔다

리포트에 OKLCH 권장값이 자동으로 제시된다. 보통 둘 중 하나로 해결된다.

첫째, semantic alias를 한 단계 어두운 값으로 변경

예를 들어 --color-danger가 raw-500을 가리키는데 흰 텍스트 대비가 부족하면, alias를 raw-600으로 변경한다. 한 줄 수정으로 모든 컴포넌트에 자동 반영된다.

둘째, 라이트/다크 동등성 위반일 때

다크 블록에서 해당 토큰을 수동으로 재정의한다. 예를 들어 다크에서 --color-text-on-brand를 흰색에서 검정으로 뒤집는다.

다크모드가 어색하다

기본 다크 토큰은 라이트의 단순 swap이라 매끄럽지 않을 수 있다. `/qa-a11y --check-dark-parity`를 돌리면 동등성 위반을 모두 잡아준다. 위반이 0건이어도 시각적으로 어색하다면 그건 색상 자체가 아닌 위계 문제일 가능성이 높다.

pnpm eval 점수가 갑자기 떨어졌다

기준선 대비 ▼가 났다면 직전 변경이 산출물 정합성을 깬 것이다. 리포트의 FAIL 항목과 위반 파일을 보고 고친다. 의도한 변경이라 새 상태가 정상이라면 `pnpm eval:baseline`으로 기준선을 갱신한다. 단, 갱신 전 FAIL이 진짜 허용 가능한지 반드시 확인할 것 — 기준선을 무심코 낮추면 회귀 추적이 무의미해진다.

Storybook이 안 뜬다

`pnpm storybook` 명령이 첫 실행에선 1~2분 걸린다. 포트 6006이 이미 사용 중이면 다른 포트로 자동 폴백된다 — 콘솔에 표시되는 URL을 따른다. 에러가 계속 나면 `node_modules`를 지우고 `pnpm install`부터 다시 한다.

Figma MCP 인증이 안 된다

Figma 유료 계정이 필요하다. 무료 계정으로는 MCP 인증이 거부된다. 인증이 자꾸 풀리면 `/mcp` 명령으로 재인증한다. 그리고 솔직히 — Figma 없이도 충분히 작동한다. 시안 자동 읽기 기능만 빠진다.

에이전트가 자기 멋대로 동작한다

에이전트 호출이 의도와 다르다면 두 가지를 확인한다. 먼저 `PROJECT.md`의 톤과 금지 규칙이 비어있는지 본다. 비어있으면 에이전트가 일반적인 기본값으로 작동한다 — 그 결과가 "AI가 만든 듯한" 느낌이다. 톤은 형용사가 아닌 구체적 명사구로 적는다. "예쁨"이 아니라 "노션처럼 정돈된 정보 밀도"같이.

다음으로 `.claude/CLAUDE.md`를 확인한다. 이 파일이 프로젝트 헌법이다. 짧게 유지되어 있지만, 프로젝트 특수 요구가 있다면 이 파일에 한두 줄 추가하는 게 가장 강력하다.

07

ARCHITECTURE

구조 더 깊이 보기

Harness Core를 단순 사용을 넘어 변형하거나 확장하려면 내부 구조를 이해해야 한다.

폴더 구조의 의미

harness-core/	
├ PROJECT.md	사람이 채움 · 한 번
├ DESIGN.md	에이전트 자동 갱신
├ .claude/	자동화 로직 전부
│ └ agents/	에이전트 11 (+figma 3종)
│ └ commands/	커맨드 9 (+figma 3종)
│ └ hooks/	혹 5 + _shared
│ └ skills/	스킬 6 (self-check 포함)
├ eval/	회귀 추적 + 혹 테스트 · v0.3+
├ src/	
│ └ tokens/	토큰 · 보호됨
│ └ components/ui/	shadcn 원본
│ └ components{이름}/	사용자 컴포넌트
│ └ wireframes/	회색 5단계만
└ docs/	QA 리포트 · ADR

FIG 06 · FOLDER TREE

harness-core 폴더 구조 — 오렌지는 폴더, 회색은 파일

네 가지 메커니즘의 분리

에이전트, 스킬, 혹, 커맨드 — 네 가지가 분리되어 있다. 각각 역할이 다르다.

AGENT — 에이전트

큰 작업을 위임받는 백그라운드 일꾼. 메인 대화창의 컨텍스트를 적게 차지하면서 자기 영역의 전문성을 가진다. ds-token-builder, qa-a11y 등.

SKILL — 스킬

재사용 가능한 절차 라이브러리. 에이전트가 호출해서 사용. 결정론적 작업(OKLCH 변환, WCAG 계산)은 .mjs 스크립트로 코드화.

HOOK — 훅

AI가 잘못된 작업을 하려고 할 때 자동으로 차단하는 가드레일. 프롬프트로 "하지 마라"가 아닌 시스템 레벨 강제. 패턴은 `_shared.mjs` 단일 소스.

COMMAND — 커맨드

사용자가 슬래시로 호출하는 단축어. `/init`, `/ds-component` 같은 것들. 주로 에이전트에게 작업 위임하는 짧은 wrapper.

자동 가드레일 5종

다음 다섯 가지 훅이 백그라운드에서 항상 작동한다. v0.3.3부터 모두 Write·Edit·MultiEdit 입력을 동일하게 검사하며, 차단은 표준 `hookSpecificOutput` 형식으로 출력한다.

훅	역할
protect-files	토큰, shadcn 원본, <code>.claude/</code> 직접 수정 차단 (경계 매칭)
detect-hardcoded	hex 컬러, Tailwind 단축 클래스, px 단위 차단
enforce-grayscale	와이어프레임에서 색상 토큰 사용 차단
check-a11y-attrs	<code>alt·aria-label·아이콘 버튼</code> 접근 이름 누락 경고 (차단 X)
notify	작업 완료 시 데스크탑 알림 (<code>execFileSync</code> , 인젝션 차단)

자동 검증의 4단계

접근성과 정합성은 네 시점에 자동으로 검증된다. v0.3에서 시간축을 따라가는 네 번째 시점(Eval 기준선)이 더해졌다.

- 1 토큰 생성 시점**
color-scale-builder — WCAG 미달 단계를 주석으로 표시
- 2 컴포넌트 생성 시점**
shadcn-wrapper — cva variant의 모든 색상 페어 검사
- 3 QA 시점**
qa-a11y — 전체 재검사 + 다크모드 동등성까지
- 4 Eval 기준선 시점**
pnpm eval — 기준선 대비 회귀 감지

FIG 6-1 · 4-LAYER VALIDATION

검증이 작동하는 네 시점 — 앞 셋은 지금 괜찮나, 넷째는 어제보다 나빠졌나를 본다

self-check — 완료 전 자동 점검

Eval을 사람이 매번 기억해 돌리지 않아도 된다. self-check 스킴은 에이전트가 "완료/done" 의사를 표현하는 순간 로딩되어, 완료 선언 전에 스스로 pnpm eval과 체크리스트로 1차 점검을 한다. 혹은 작업 중 "막는" 것이라면, self-check는 마무리 시점에 "스스로 돌아보는" 것이다.

PROJECT.md 깊이 채우기

기본 정보만 채워도 작동하지만, 톤과 금지 규칙을 채워두면 결과 품질이 크게 달라진다. 톤은 형용사 대신 구체적 비교 대상을 적는다 — "미니멀"이 아니라 "노션처럼 정돈된 정보 밀도"같이. 금지 규칙은 디자인의 캐릭터를 만드는 가장 강한 도구다. "그림자 사용 최소화", "라디우스 12px 초과 금지", "강조는 색이 아닌 typography" 같은 명시적 제약이 "AI가 만든 듯한" 느낌을 가장 효과적으로 제거한다.

커스터마이징할 만한 곳

프로젝트 특수 요구가 생기면 위에서부터(가벼우면서 효과 큰 순서) 검토한다.

-
- 1 **PROJECT.md의 "톤"과 "금지 규칙" 추가**
가장 가볍고 효과 큼
 - 2 **.claude/CLAUDE.md에 한두 줄 추가**
모든 에이전트가 읽는 헌법
 - 3 **기존 에이전트의 .md 파일 수정**
작업 절차 자체 변경
 - 4 **새 스킬 추가**
반복 작업이 있을 때
 - 5 **새 훅 추가**
새 금지 규칙을 강제할 때 (패턴은 _shared.mjs에)
 - 6 **새 Eval 태스크 추가**
새 회귀를 추적할 때 (eval/lib + tasks + fixture)

08

LIMITATIONS

한계와 미해결

이 도구는 만능이 아니다. 의도적으로 안 한 것과 못한 것이 있다. 사용 전에 알아두면 기대 조정에 도움이 된다.

Figma MCP: read-only가 기본, write는 조건부

v0.3.3까지는 "Figma MCP는 read-only"가 전제였다. v0.4부터는 조건부다 — write 가능한 Figma 플러그인 MCP(use_figma)가 연결돼 있으면 Figma에 컴포넌트·토큰을 직접 생성하고, 없으면 기존처럼 read-only로 폴백한다. 같은 하네스가 환경에 따라 두 방식을 자동으로 오간다.

환경	컴포넌트	토큰	Figma 화면 QA
write MCP 있음	/build-component (실제 생성)	/figma-tokens (실제 생성)	/qa-figma-wireframe
read MCP만	스펙시트 폴백	/ds-token (비교)	/qa-figma-wireframe
Figma 없음	코드만 (/ds-component)	코드만 (/init)	/qa wireframe (코드)

와이어 → 디자인 자동 변환은 없다

와이어프레임을 색이 들어간 디자인으로 자동 변환하는 단계는 의도적으로 빠뜨렸다. 그 단계는 사람의 판단이 가장 중요하다 — 어디에 강조를 둘지, 어떤 상태를 우선시할지. 현재는 wireframes/{이름}.tsx를 components/{이름}/로 복사한 뒤 토큰을 입히는 수동 작업이다.

Eval은 아직 "가벼운" 형태

v0.3의 Eval은 산출물을 1회 정적 채점한다. "10번 중 8번 성공" 같은 확률적 성공률은 아직 측정하지 않는다. 결정론적인 디자인 토큰 영역에서는 정적 채점으로 충분하지만, 에이전트의 생성 안정성 자체를 재려면 claude -p 루프로 감싸는 확장이 필요하다. 채점 로직은 그대로 재사용된다.

아직 검증되지 않은 영역

코드·혹·Eval은 typecheck·build·fixture·혹 회귀 테스트로 자동 검증된다. 다만 다음은 실제 환경에서 폴사이클을 돌려봐야 확인되는 영역이다: Storybook 실구동, INSTALL.sh의 pnpm·Playwright 설치, write Figma MCP 실연결 시의 라이브 생성(컴포넌트·토큰·화면), 그리고 에이전트 지시가 실제 Claude Code에서 의도대로 실행되는지. 파일 정합성·eval·hook·regression은 자동 검증되지만, 라이브 Figma 생성은 사용자 환경(Figma 유료 + 플러그인)에서 확인이 필요하다.

Figma 트랙을 쓰기 위한 전제

/build-component/figma-tokens/qa-figma-wireframe를 실제로 돌리려면 다음이 필요하다. 하나라도 없으면 에이전트는 안내 메시지를 출력하고 폴백 경로(스펙시트·비교·코드 검증)로 돌린다.

Figma 유료 계정

MCP 인증에 필요

write 가능 Figma 플러그인 MCP

use_figma 연결 (부록 A-2 절차)

PROJECT.md 작성

Figma 파일 키 + 페이지 ID(Foundations/Components/Documentation/Wireframes)

read 전용 작업

/ds-token/qa-figma-wireframe은 read MCP만 있어도 동작

APCA는 미지원

접근성 검증은 WCAG 2.2 알고리즘을 사용한다. WCAG 3 후보인 APCA는 더 정확하지만 아직 표준이 아니라 포함하지 않았다. 필요하면 별도 도구를 병행한다.

Windows 미지원

INSTALL.sh는 bash 스크립트라 Windows에서 직접 실행되지 않는다. WSL 사용을 권장한다.

"AI가 만든 듯한" 결과의 완전 해결은 아님

PROJECT.md의 톤과 금지 규칙으로 상당 부분 완화되지만, 특정 산업이나 브랜드 정체성을 깊이 반영하려면 결국 사람의 개입이 필요하다. 이 도구는 출발점을 단단하게 만드는 데 강하지, 최종 결과의 "맛"을 결정하지는 못한다.

V0.2 → V0.5에서 해소된 것

v0.2의 미해결은 "하네스 개선 효과를 수치로 증명할 수단이 없음"이었고, v0.3 Eval 기준선이 이를 해결했다. v0.3.3은 가드레일의 "켜져 보이지만 작동 안 하는" 결함을 제거하고 회귀 테스트로 재발을 막았다. v0.4~v0.5는 가장 큰 전제였던 "Figma는 read-only"를 깨고, write MCP가 있으면 컴포넌트·토큰을 실제로 생성하고 Figma 화면까지 QA하도록 확장했다.



APPENDIX

설치와 연결

본문 02장은 harness-core 압축을 푼 뒤를 다룬다. 이 부록은 그 전 단계 — Claude Code 설치와 Figma 연결 — 을 따로 모았다. 처음 환경을 잡을 때 한 번만 보면 된다.

A-1. Claude Code 설치

Claude Pro·Max·Teams·Enterprise·API 크레딧 계정 중 하나가 필요하다. 무료 플랜으로는 안 된다. 설치 방법은 세 가지다.

방법 1 — 네이티브 인스톨러 (권장)

Node.js가 필요 없고 백그라운드에서 자동 업데이트된다. Anthropic이 기본으로 지원하는 방법.

```
# macOS / Linux
curl -fsSL https://claude.ai/install.sh | bash
# Windows (PowerShell)
irm https://claude.ai/install.ps1 | iex
```

방법 2 — Homebrew (Mac에서 brew를 쓴다면)

```
brew install --cask claude-code
# 최신 기능을 빨리 받고 싶으면 (롤링 채널)
brew install claude-code@latest
```

Homebrew 설치는 자동 업데이트가 안 된다. 새 버전은 `brew upgrade claude-code`로 직접 받아야 하고, cask 특성상 출시 후 며칠 지연될 수 있다.

방법 3 — npm

```
npm install -g @anthropic-ai/claude-code
```

Node.js 18 이상이 필요하다. harness-core는 20 이상을 요구하니 20+ 를 권장한다. 권한 오류가 나도 sudo는 쓰지 마라 — nvm으로 Node를 홈 디렉토리에 설치하는 게 올바른 해결이다.

설치 확인 + 첫 로그인

```
claude --version  
claude # 첫 실행 시 브라우저로 로그인
```

VS Code·Cursor 사용자는 Marketplace에서 "Claude Code" 확장을 설치하면 에디터 안에서 바로 쓸 수 있다.

A-2. Figma MCP 연결

harness-core는 Figma 없이도 작동한다. 토큰 양방향 동기화(/ds-token)나 시안 읽기를 쓸 때만 필요하다. Claude Code 설치와 로그인에 끝났다면 아래 6단계를 순서대로 진행한다. 총 5~10분 걸린다.

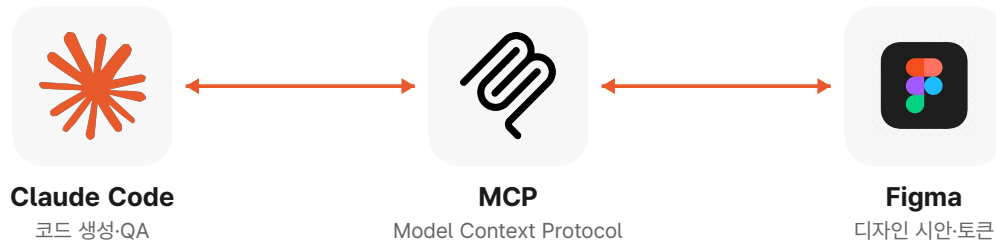


FIG A2 · CONNECTION

Claude Code와 Figma를 MCP가 잇는다 — 양방향으로 시안과 토큰이 오간다

- 1 앱 다운로드**
mcp.metadata.co.kr 에서 OS에 맞는 앱 설치
- 2 코드 복사**
앱 Setting에서 Claude Code용 연결 코드 복사
- 3 Claude Code 붙여넣기**
코드 붙여넣고 "talk to figma mcp 연결해줘" 요청
- 4 Start Server**
MCP 앱에서 서버 실행
- 5 Figma 플러그인 실행**
플러그인 실행 후 표시되는 채널 코드 복사
- 6 채널 연결**
복사한 채널 코드를 Claude Code에 입력

확인 포인트

Claude Code가 MCP 서버를 인식했다는 응답을 보여주면 정상이다. 오류가 나면 코드가 완전히 복사됐는지(줄 끊김 없는지) 먼저 확인한다.

알아둘 점

Figma 연동은 선택이다. 연결하지 않아도 harness-core의 토큰 생성·컴포넌트·와이어프레임·QA·Eval은 모두 작동한다. Figma를 붙이면 시안을 읽어 코드로 옮기거나 토큰을 양방향으로 맞추는 작업이 추가될 뿐이다.

이 매뉴얼은 v0.5.0 기준이다. 변경 사항은 CHANGELOG.md에서 확인할 수 있고, 버그나 개선 제안은 프로젝트 저장소의 Issues에 남기면 된다.